

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquelos.

Definimos 3 mini-ciclos, agrupando todo lo relacionado a cada funcionalidad en un mismo ciclo:

- **Mini-ciclo 1:**
 - Se eliminó el uso de abstract en StackingItem convirtiéndola en una clase normal, y se ajustaron Cup y Lid para usar el constructor de la superclase correctamente.
- **Mini-ciclo 2:**
 - Se implementaron los 4 métodos nuevos: Tower(cups), swap, cover y swapToReduce, junto con la actualización del diagrama de clases.
- **Mini-ciclo 3:**
 - Se diseñaron y ejecutaron los casos de prueba de la clase TowerC2Test en modo invisible.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

Los 3 mini-ciclos fueron completados. El proyecto cumple con todos los requisitos funcionales del ciclo 2: crear torre con número de tazas, intercambiar objetos, tapar tazas y consultar un intercambio que reduzca la altura.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

Cortes-19 horas

Cubides-19 horas

4. ¿Cuál consideran que fue el mayor logro? ¿Por qué?

El mayor logro fue implementar correctamente los 4 métodos nuevos manteniendo el código simple y comprensible para nuestro nivel actual. Logramos que el simulador funcione de forma completa sin usar conceptos avanzados que aún no dominamos.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue eliminar el uso de abstract en StackingItem. Al principio no entendíamos cómo hacer que Cup y Lid siguieran funcionando sin métodos abstractos. Lo resolvimos convirtiendo StackingItem en una clase normal con atributos y constructor propios, y haciendo que las subclases llamaran a super() para inicializarse.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Trabajamos bien como equipo, nos comunicamos constantemente y nos apoyamos cuando alguno tenía dudas. Nos comprometemos a planificar mejor los mini-ciclos desde el inicio del próximo ciclo para no acumular trabajo al final.

7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿Por qué?

La práctica más útil fue el diseño simple. Nos ayudó a no complicar el código con soluciones muy avanzadas, sino a buscar la forma más clara y directa de implementar cada método, lo que hizo el código más fácil de entender y mantener.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

- Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.
- Oracle. (2024). *Java SE Documentation*. <https://docs.oracle.com/en/java/>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- OpenAI. (2024). *ChatGPT* (versión GPT-4). <https://chat.openai.com>

La más útil fue la documentación oficial de Java, ya que nos ayudó a entender cómo funciona la herencia y los constructores con `super()`.